

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – 5

Roll No.: T017	Name: Radhika Mali
Class: TYIT	Batch: 1
Date of Assignment: 28/07/2026	Date/Time of Submission: 28/07/2026

a) Create a web application to demonstrate the use of different types of Cookies.

Algorithm:

1. Start.
2. Enter user data.
3. Create and store cookies.
4. Read and display cookies.
5. Update or delete cookies if required.
6. Stop.

Code:

webform1.aspx.cs:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;
namespace WebApplication1
{
    public partial class WebForm1 : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            if(IsPostBack)
            {
                int ViewStateVal =
Convert.ToInt32(ViewState["count"]) + 1;
                Label2.Text = "View Sate =
"+ViewStateVal.ToString();
            }
            else
            {
                ViewState["count"] = 1;
            }
        }
        protected void Button1_Click(object sender,
EventArgs e)
        {
```

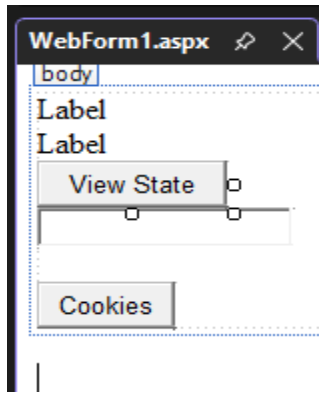
```
                Label1.Text = ViewState["count"].ToString();
            }
        }
        protected void Button2_Click(object sender,
EventArgs e)
        {
            HttpCookie h = new HttpCookie("name");
            h.Value = TextBox1.Text;
            Response.Cookies.Add(h);
            Response.Redirect("WebForm2.aspx");
        }
    }
}
```

webform2.aspx.cs:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;
namespace WebApplication1
{
    public partial class WebForm2 : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            if (Request.Cookies["name"]!=null)
            {
```

```
        Response.Write("Welcome " + Request.Cookies["name"].Value);  
    }  
}
```

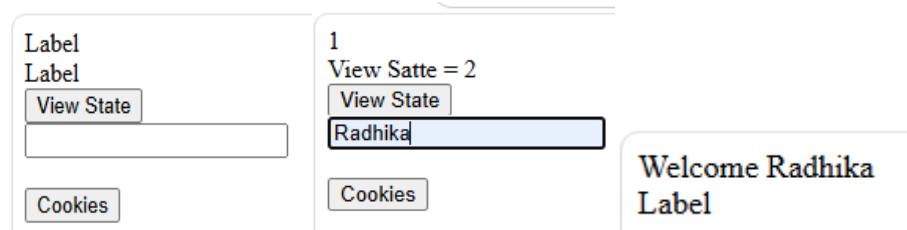
Design:



webform2.source:

```
<title></title>  
</head>  
<body>  
    <form id="form1" runat="server">  
        <div>  
            <asp:Label ID="Label1" runat="server" Text="Label"></asp:Label>  
        </div>  
    </form>  
</body>  
</html>
```

Output:



b) Create a Product Table in SQL Server and perform Insert Operation using Entity Framework Core.

Algorithm:

1. Start.
2. Create a .NET project.
3. Add EF Core packages.
4. Create the **Product** model and **DbContext**.
5. Configure the SQL Server connection.
6. Run migrations to create the table.
7. Insert a product using **Add()** and **SaveChanges()**.
8. Verify the record.
9. Stop.

Code:

Step 1: Create Database

```
CREATE DATABASE ProductDB;  
GO  
USE ProductDB;  
No need to create the table manually. EF Core will create it.
```

Step 2: Product Model (Product.cs)

```
public class Product  
{  
    public int Id { get; set; }  
    public string Name { get; set; }  
    public int Price { get; set; }  
}
```

Step 3: AppDbContext (AppDbContext.cs)

```
using Microsoft.EntityFrameworkCore;
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext>
options)
: base(options)
{
}
public DbSet<Product> Products { get; set; }
}
```

Step 4: appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection":
      "Server=(localdb)\MSSQLLocalDB;Database=ProductDB
;Trusted_Connection=True;TrustServerCertificate=True;"
  }
}
```

If you use SQL Server Express:

```
"Server=.\SQLEXPRESS;Database=ProductDB;Trusted_
Connection=True;TrustServerCertificate=True;"
"
```

Step 5: Program.cs

```
using Microsoft.EntityFrameworkCore
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();
builder.Services.AddDbContext<AppDbContext>(options
=>
options.UseSqlServer(
builder.Configuration.GetConnectionString("DefaultConne
ction")));
var app = builder.Build();
app.UseStaticFiles();
app.UseRouting();
app.MapControllerRoute(
name: "default",
pattern: "{controller=Home}/{action=Index}/{id?}");
app.Run();
```

Output:

Product Inserted Successfully

	Id	Name	Price
1	1	Laptop	50000

Step 6: HomeController

```
using Microsoft.AspNetCore.Mvc;
public class HomeController : Controller
{
    private readonly AppDbContext _context;

    public HomeController(AppDbContext context)
    {
        _context = context;
    }
    public IActionResult Index()
    {
        Product p = new Product();
        p.Name = "Laptop";
        p.Price = 50000;
        _context.Products.Add(p);
        _context.SaveChanges();
        return Content("Product Inserted Successfully");
    }
}
```

Step 7: Install Packages

Open Package Manager Console

Install-Package Microsoft.EntityFrameworkCore.SqlServer

Install-Package Microsoft.EntityFrameworkCore.Tools

Step 8: Migration

Add-Migration InitialCreate

Update-Database

Step 9: Run

Run the project and open:

<https://localhost:xxxx/Home/Index>

Output:

Product Inserted Successfully

Step 10: Verify

Open SQL Server and run:

SELECT * FROM Products;

c) Develop a web application to perform CRUD Operations (Create, Read, Update, Delete) on the Employee table using EF Core.

Algorithm:

Steps :

1. Define Model: Create Employee.cs with properties (Id, Name, Department, Salary).
2. Setup DbContext: Create ApplicationDbContext.cs with DbSet<Employee> Employees and register it inside Program.cs.
3. Apply Migrations: Run Add-Migration AddEmployeeTable followed by Update-Database in Package Manager Console.
4. Build Controller Actions: In EmployeesController.cs, implement methods for Create, Read, Update, and Delete operations.
5. Create Views: Generate corresponding Razor views under Views/Employees/ (Index, Create, Edit, Delete).

Code:

Employee.cs
namespace EmployeeCRUD.Models

```
{
    public class Employee
    {
        public int Id { get; set; }
        public required string Name { get; set; }

        public required string Department { get; set; }

        public double Salary { get; set; }
    }
}
```

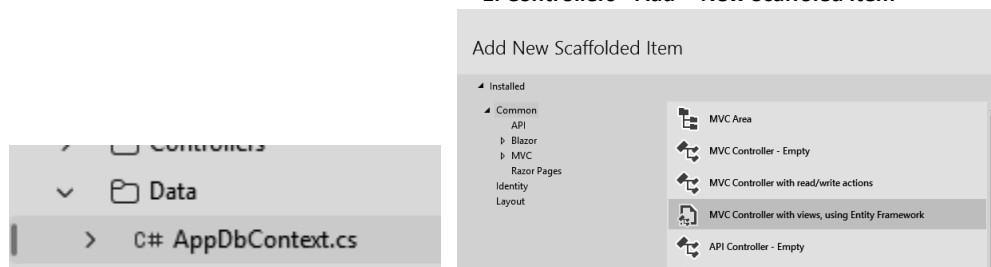
Program.cs

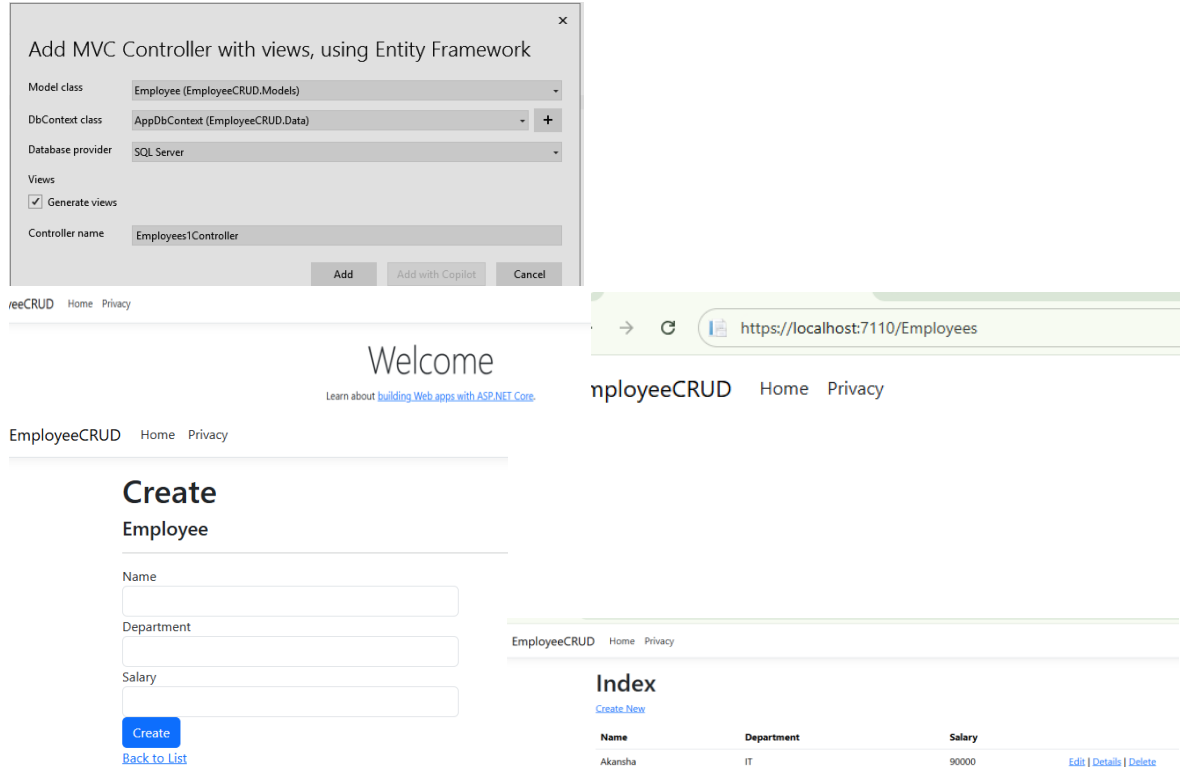
```
using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Data;
builder.Services.AddControllersWithViews();
builder.Services.AddDbContext<AppDbContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("MyConnection")));
appsettings.json:
"AllowedHosts": "*",
```

```
"ConnectionStrings": {
    "MyConnection":
    "Server=(localdb)\MSSQLLocalDB;Database=EmployeeDB;Trusted_Connection=True;TrustServerCertificate=True",
    "AppDbContext":
    "Server=(localdb)\mssqllocaldb;Database=AppDbContext;Trusted_Connection=True;MultipleActiveResultSets=true"
}
```

```
AppDbContext.cs
using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Models;
namespace EmployeeCRUD.Data
{
    public class AppDbContext:DbContext
    {
        public
        AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
        public DbSet <Employee> Employees { get; set; }
    }
}
```

1. Controllers->Add-> New Scaffolded Item



Output:**d) Authentication and Authorization in ASP.NET Core.****Algorithm:**

1. Create an ASP.NET Core MVC project.
2. Enable Cookie Authentication in Program.cs.
3. Create Login action in HomeController.
4. Verify username and password.
5. Redirect to Secure page after successful login.
6. Use [Authorize] to protect the Secure page.
7. Logout and redirect to Login.

Code:**Program.cs**

```
using Microsoft.AspNetCore.Authentication.Cookies;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();
builder.Services.AddAuthentication(CookieAuthentication
Defaults.AuthenticationScheme)
.AddCookie(options =>{
options.LoginPath = "/Home/Index";
});
builder.Services.AddAuthorization();
var app = builder.Build();
```

```
app.UseStaticFiles();
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllerRoute(
name: "default",
pattern:
"/{controller=Home}/{action=Index}/{id?}");
app.Run();
```

HomeController.cs

```
using Microsoft.AspNetCore.Mvc;
```

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;
using System.Security.Claims;
namespace AuthDemo.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }
        [HttpPost]
        public async Task<IActionResult> Index(string
username, string password)
        {
            if (username == "admin" & password == "123")
            {
                var claims = new List<Claim>
                {
                    new Claim(ClaimTypes.Name, username)
                };
                var identity = new ClaimsIdentity(claims,
CookieAuthenticationDefaults.AuthenticationScheme);
                await HttpContext.SignInAsync(
CookieAuthenticationDefaults.AuthenticationScheme,
new ClaimsPrincipal(identity));
                return RedirectToAction("Secure");
            }
            ViewBag.Message = "Invalid Username or
Password";

```

```

return View();
}
[Authorize]
public IActionResult Secure()
{
    return Content("Welcome! Login Successful.");
}
public async Task<IActionResult> Logout()
{
    await HttpContext.SignOutAsync(
CookieAuthenticationDefaults.AuthenticationScheme);
    return RedirectToAction("Index");
}
}

```

Views/Home/Index.cshtml

```

<h2>Login</h2>
<form method="post">
    Username :
    <input type="text"
name="username" /><br />
    Password :
    <input type="password"
name="password" /><br />
    <input type="submit"
value="Login" />
</form>
<p style="color:red">@ViewBag.Message</p>

```

Output:

Login

Username:

Password:

Login

Welcome! Login Successful.